



Operations Runbook Brainbox, day to day

What runs where, how to give a company its keys and staff accounts, how to share the Odoo add-on, how code reaches production, how to back up, publish the SDKs, move to a new server, and what to do when something breaks.

SERVER

165.227.77.33

PUBLIC URL

port.smartpowerbilling.com

DATE

9 October 2026

Written for the project owner. Each section answers one question in plain language first, then gives the exact commands. Commands marked "on the server" are run over SSH on the brainBox server.

Contents

1	What runs where	The server, the containers, GitHub, npm, Odoo, the dashboard	3
2	A company wants keys	Onboarding, what to send, what never to send, revoking	4
3	Staff and the dashboard	First owner, logging in, hosting, invites, unanswered questions	7
4	Training the AI	Where and with which credentials	9
5	Sharing the Odoo add-on	The zip, installation, settings, updating the SDK, local test	10
6	Deployments and backups	push → GitHub Actions → deploy.sh; migrations; backup and restore	12
7	SDK releases	What is published, what is pending, how to publish	14
8	Moving to a new server	bootstrap-server.sh, restore, DNS, HTTPS, performance	16
9	Security checklist	Firewall, ports, secrets, SSH, rotation	19
10	Troubleshooting	Health checks, logs, slow answers, 401 / 403, missing model	20
11	Quick reference card	The commands you use most, on one page	22

THE OTHER TWO GUIDES

This runbook is about **operating** the service. It does not repeat the two existing guides; it points to them by section:

- **Odoo guide** = *Brainbox AI for Odoo — How it works* ([Brainbox-Odoo-Integration-Guide.pdf](#)): architecture, keys, roles and every Odoo setting.
- **Training & API reference** = *Brainbox AI — Training Guide & API Reference* ([Brainbox-Training-and-API-Reference.pdf](#)): training, knowledge gaps, staff roles and every endpoint.

ABOUT SECRETS IN THIS DOCUMENT

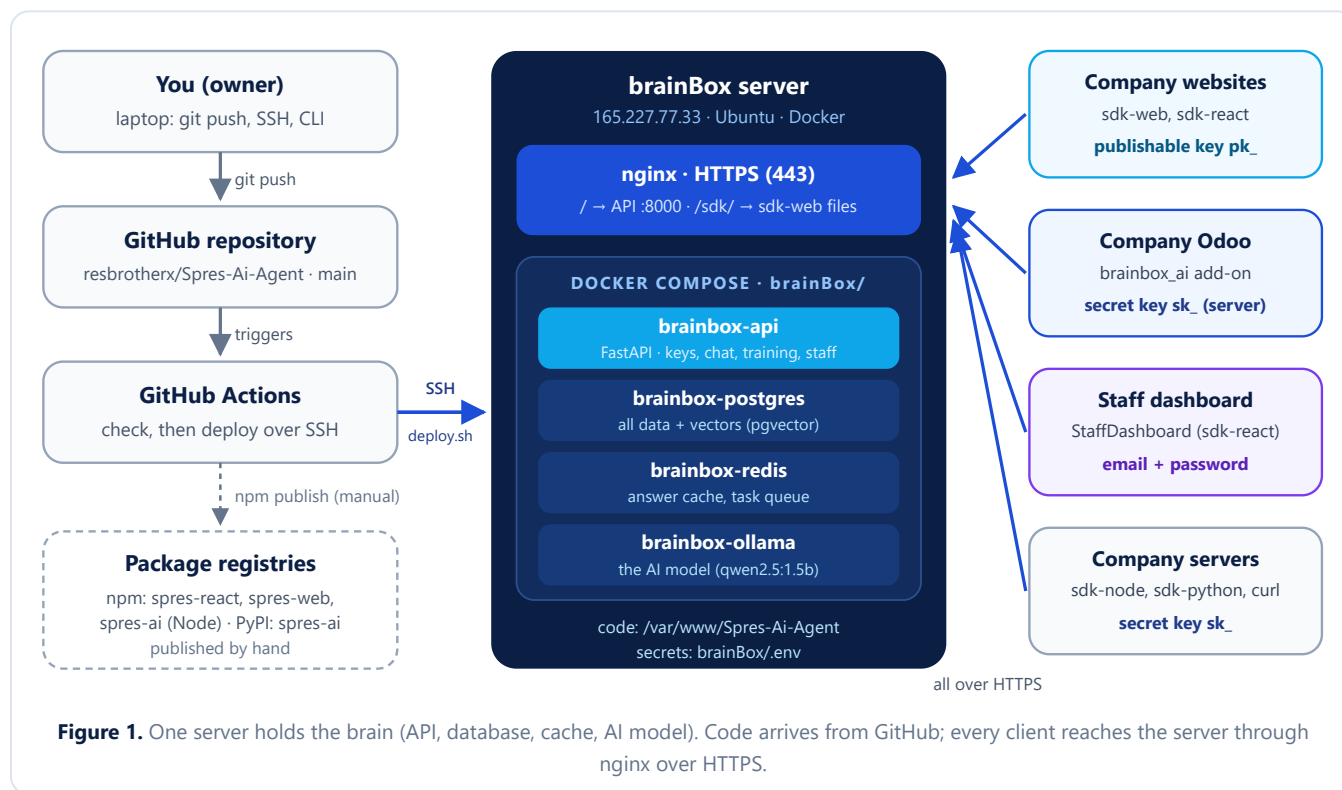
No real key, password or token appears here. Keys are written as `pk_live_XXX` / `sk_live_XXX`, the admin token as `$ADMIN_TOKEN`, and the example company is `acme`. The real secrets live only in `/var/www/Spres-Ai-Agent/brainBox/.env` on the server and in your password manager.

NEW FILES THAT COME WITH THIS RUNBOOK

`deploy/bootstrap-server.sh` (sets up a new server), `deploy/README.md` (deploy scripts, GitHub secrets, deploy key) and `docs/brainbox_ai-odoo17-v2.0.0.zip` (the Odoo add-on, ready to send to a company).

1 What runs where

Everything that thinks lives on **one server** that you control. Everything else (websites, Odoo, the staff dashboard, scripts) is a client that talks to that server over HTTPS with a key or a staff login.



Piece	Where it lives	What it does · how it is updated
brainbox-api	Docker container, port 8000, code bind-mounted from <code>brainBox/</code>	The backend: keys, chat, training, staff dashboard API. Updated by every deploy (restart).
brainbox-postgres	Docker container, volume <code>brainbox_postgres_data</code> , only on 127.0.0.1:5432	Every company's documents, vectors, chats, keys, staff. The one thing to back up.
brainbox-redis	Docker container, internal only	Answer cache and task queue. Can be lost safely.
brainbox-ollama	Docker container, internal only, volume <code>brainbox_ollama_data</code>	Runs the AI model named in <code>OLLAMA_MODEL</code> . Model files are re-downloadable.
nginx	On the server (site <code>resai</code> today)	HTTPS for <code>port.smartpowerbilling.com</code> , forwards to :8000, serves <code>/sdk/</code> from <code>sdk-web/</code> .
GitHub + Actions	<code>resbrotherx/Spres-Ai-Agent</code> , workflow <code>deploy.yml</code>	Source of truth. A push to <code>main</code> deploys automatically (section 6).
npm / PyPI packages	npmjs.com, pypi.org	SDKs other developers install. Published by hand (section 7).
Odoo add-on	Each company's own Odoo server	<code>brainbox_ai</code> , sent as a zip (section 5).
Staff dashboard	A web page you host (section 3)	Where each company's staff train the AI, see unanswered questions and manage keys.

2 A company wants keys

Onboarding a company takes five minutes: pick a name for it, give it a first owner, and let that owner (or you) create its keys.

IN PLAIN WORDS

Think of the server as the head office. Companies never get access to the head office itself: they get **keys** (for their websites and systems) and **staff accounts** (for their people). Each key and account belongs to exactly one company, called a **tenant**, and can only see that company's knowledge.

- 1 Choose the tenant id.** A short, lowercase name with no spaces, e.g. `acme`. Nothing needs to be "created": the tenant exists as soon as a key or staff account uses the name.
Case-sensitive free text. A typo (`Acme` vs `acme`) silently makes a second, empty company, so write it down and copy-paste it. Check existing names with `keys list` / `staff list`.
- 2 Create their first owner.** One person at the company who will manage everything else. Either from the server CLI or over HTTPS with the admin token (commands below). Without a password you get an **invite link** valid for 7 days.
- 3 Send the owner the invite link and the dashboard address.** They set their password and log in to the staff dashboard (section 3).
- 4 Keys: they create their own, or you do.** In the dashboard: **Settings** → **API keys** (admins and owners). Publishable key for websites and apps, secret key for servers and Odoo. Or you create them with the CLI / admin API.
- 5 Send the connection details** (list below) and confirm the key works: `curl -H "X-API-Key: sk_live_xxx" https://port.smartpowerbilling.com/api/health` answers with the key's type and tenant.

Create the first owner

ON THE SERVER (CLI INSIDE THE API CONTAINER)

```
# invite link (valid 7 days) - send it to the person
docker exec brainbox-api python -m app.cli staff create --tenant acme --email owner@acme.com --role owner --name "Ada Lovelace"

# or set a password now (you are prompted twice, nothing is echoed)
docker exec -it brainbox-api python -m app.cli staff create --tenant acme --email owner@acme.com --role owner --password

docker exec brainbox-api python -m app.cli staff list --tenant acme
```

FROM ANYWHERE, WITH THE ADMIN TOKEN (BRAINBOX_ADMIN_TOKEN FROM THE SERVER'S .ENV)

```
curl -X POST https://port.smartpowerbilling.com/api/admin/staff \
-H "X-Admin-Token: $ADMIN_TOKEN" -H "Content-Type: application/json" \
-d '{"tenant_id":"acme","email":"owner@acme.com","full_name":"Ada Lovelace","role":"owner"}'
# → {"user": {...}, "invite_url": "https:// ... /#/accept-invite?token=... "}
```

Create keys yourself (optional)

```
# the raw key is printed ONCE – copy it into your password manager straight away
docker exec brainbox-api python -m app.cli keys create --tenant acme --type publishable --name "Acme website"
docker exec brainbox-api python -m app.cli keys create --tenant acme --type secret --name "Acme Odoo" --
expires 2027-12-31
docker exec brainbox-api python -m app.cli keys list --tenant acme          # never shows raw keys

# same over HTTPS
curl -X POST https://port.smartpowerbilling.com/api/admin/keys \
-H "X-Admin-Token: $ADMIN_TOKEN" -H "Content-Type: application/json" \
-d '{"tenant_id": "acme", "key_type": "secret", "name": "Acme Odoo"}'      # → { ..., "api_key": "sk_live_..." }
```

Which key goes where

Used in	Key	Notes
sdk-web on a website	<code>publishable pk_</code>	Pasted in the page; visible to visitors, which is fine: it can only chat, and only as <code>public</code> .
sdk-react ChatWidget / ChatPanel	<code>publishable pk_</code>	Same rules as sdk-web. Mobile apps (sdk-react-native) too.
Odoo add-on settings	<code>secret sk_</code>	Stored on the Odoo server only; lets Odoo say which role each user has.
sdk-node, sdk-python, curl, back-ends	<code>secret sk_</code>	Server-side code only. Never in a browser or app.
Training by API, TrainingPanel	<code>secret sk_</code> or staff login	<code>/api/train/*</code> and <code>/api/ingest</code> need a secret key or a trainer+ staff login.
Staff dashboard	staff login	Email + password. No key at all.
Admin API (<code>/api/admin/*</code>)	<code>admin token</code>	Yours alone. Works for every tenant. Never sent to a company.

The full explanation of key types, prefixes and what each can call is in the Training & API reference, section 7 "Keys explained", and the Odoo guide, section 4.

SEND THE COMPANY

- Backend URL: `https://port.smartpowerbilling.com`
- Their tenant id, e.g. `acme`
- Dashboard link + the owner's invite link
- Publishable key (websites, apps)
- Secret key (Odoo, servers) — through a password manager share or another secure channel, not in a normal email or chat
- Script tag:
`https://port.smartpowerbilling.com/sdk/brainbox-web-sdk.js`
- The Odoo add-on zip (if they use Odoo)

NEVER SEND

- `BRAINBOX_ADMIN_TOKEN` (opens every company)
- `JWT_SECRET_KEY` (lets anyone forge staff logins)
- `POSTGRES_PASSWORD` or the `.env` file
- The server's SSH password or keys
- Another company's keys or tenant id
- A secret key for use in a website or mobile app — give a publishable key instead

Revoking and rotating keys

A revoked key stops working on the next request. To **rotate** (replace a key without downtime): create the new key, let the company install it, check it works, then revoke the old one. Each key shows *last used*, so you can see when the old one went quiet.

```
docker exec brainbox-api python -m app.cli keys list --tenant acme      # find the id (ID column)
docker exec brainbox-api python -m app.cli keys revoke 42
curl -X DELETE https://port.smartpowerbilling.com/api/admin/keys/42 -H "X-Admin-Token: $ADMIN_TOKEN"
```

Company admins can do the same themselves in **Settings** → **API keys** (they only see their own tenant's keys). If a secret key leaks, revoke it first and ask questions later: the only effect is that their integration stops until the new key is installed.

3 Staff and the dashboard

Staff are the people at a company who look after their assistant: they train it, read what users asked, answer the questions it couldn't, and manage keys and colleagues. They all use one web page: the **staff dashboard**.

What you need

- ☐ An **owner** (or admin) account for the company — created by you as in section 2. Everyone else is invited from the dashboard by that person.
- ☐ A **hosted dashboard** (below), and `DASHBOARD_URL` in the server's `.env` pointing to it, so links in invite and password-reset emails open the right page.
- ☐ For emails (invites, alerts): the `SMTP_*` settings in `.env`. Without them, everything still works but the invite link must be copied and sent by hand (the dashboard shows it).

Where staff log in

The dashboard is the `StaffDashboard` component of the React SDK (`spres-react`). Today it runs from the `sdk-react` **preview app** on your machine:

```
cd ai/sdk-react/preview
npm install
npm run dev                # then open http://localhost:4173/#/staff
```

That is fine for you, but companies need a real address. Two ways to host it:

A. Build the preview and serve it with nginx

Quickest. Build with the server URL and serve the static files, e.g. at

`https://port.smartpowerbilling.com/dashboard/`:

```
# preview/.env.local:
VITE_BRAINBOX_API_URL=https://port.smartpowerbi
lling.com
npx vite build --base=/dashboard/
# nginx (same server block)
location /dashboard/ {
    alias /var/www/brainbox-dashboard/;
}
```

Copy `preview/dist/` to `/var/www/brainbox-dashboard/`. Staff open `.../dashboard/#/staff`.

B. A small app of your own

Cleaner for customers (no chat demo pages). Any React app:

```
import { StaffDashboard } from 'spres-react';

export default function App() {
  return <StaffDashboard
    apiUrl="https://port.smart..."
    brandName="Brainbox" />;
}
```

Build it and host the files anywhere (nginx, Netlify...). Needs `spres-react` 1.1.0, which is not on npm yet (section 7) — until then build it from the repo.

Then set `DASHBOARD_URL` and restart the API:

```
# /var/www/Spres-Ai-Agent/brainBox/.env
# option A (the preview forwards #/accept-invite links to #/staff/...)
DASHBOARD_URL=https://port.smartpowerbilling.com/dashboard
docker restart brainbox-api
```

NEVER BUILD A SECRET KEY INTO THE DASHBOARD

Anything in a `VITE_...` variable ends up inside the JavaScript files that every visitor downloads. The dashboard does not need a key: staff log in with email and password.

Inviting staff and roles

In the dashboard: **Staff** → **Invite**, enter email, name and role. The person receives an email (or the admin copies the link shown) and sets a password within 7 days. Roles, from most to least powerful:

Role	Can
owner	Everything, including managing admins and other owners. A company always keeps at least one active owner.
admin	Invite / change / remove trainers and viewers, change settings, manage API keys.
trainer	Train the AI, answer / dismiss / reopen knowledge gaps.
viewer	Read everything: overview, gaps, conversations, training sources, staff list, settings.

Endpoints behind the dashboard: `POST /api/staff/login`, `POST /api/staff/invite` (admin+), `PATCH / DELETE /api/staff/{id}`, `POST /api/staff/{id}/resend-invite`, `POST /api/staff/accept-invite`, `/forgot-password`, `/reset-password`, `GET /api/staff`. Details: Training & API reference section 6 "Staff roles" and section 10. One email address = one staff account in one company.

```
# server-side helpers when someone is locked out
docker exec -it brainbox-api python -m app.cli staff set-password owner@acme.com
docker exec brainbox-api python -m app.cli staff deactivate someone@acme.com # immediate
```

Unanswered questions (knowledge gaps)

- 1 The assistant can't answer.** No matching knowledge, a low-confidence match, the model said it doesn't know, the model was unavailable, or a user gave a thumbs-down.
- 2 It appears in the dashboard** under **Knowledge gaps**, grouped by question with a counter of how often it was asked.
- 3 Staff are notified.** An in-app notification (bell) for each active staff member, and an email if SMTP is configured on the server, the staff member has email alerts on, and the company allows it (**Settings** → **Alerts**). Emails are capped per hour (`EMAIL_MAX_PER_HOUR`, default 20).
- 4 A trainer answers it** with **Answer & train**: the answer becomes a training source, the gap is marked resolved, and the next user gets the new answer.
Full flow: Training & API reference, section 5. Email needs `SMTP_HOST` + `SMTP_FROM` (or `SMTP_USER`); also `SMTP_PORT`, `SMTP_PASSWORD`, `SMTP_TLS` = starttls / ssl / none.

4 Training the AI

Training means giving a company's assistant its knowledge: files, pasted text, or a connection to another system (for example a support desk). Each company trains its own assistant; you rarely need to do it for them.

Where	Who / credentials	How
Staff dashboard → Training	staff with role trainer, admin or owner	Upload files, paste text, connect an API, pick the audience, re-sync or delete sources. The easy way.
Knowledge gaps → Answer & train	trainer+	Answer one unanswered question; it becomes a training source.
API <code>/api/train/*</code>	secret key <code>sk_</code> or a trainer+ staff login	<code>POST /api/train/file</code> , <code>/train/text</code> , <code>/train/api-source</code> , <code>GET /api/train/sources</code> , <code>PATCH / DELETE /api/train/sources/{id}</code> , <code>POST /api/train/sources/{id}/sync</code> .
<code>TrainingPanel</code> (sdk-react)	secret key or staff token	The same training screen embedded in a company's own internal tool.

THE ONE RULE TO REMEMBER: AUDIENCES

Every source has an **audience** (public, customer, vendor, internal, admin) that decides who may get answers from it. New sources default to **internal**. Website visitors (publishable keys) only ever see **public** knowledge, so a company that wants its website bot to answer from a document must mark that document **public**.

Where to read more, in the **Training & API reference**:

Section	Topic
1	How Brainbox learns — from a PDF to an answer, in plain words
2	Ways to train — dashboard, API, answering gaps; file types and limits
3	Connecting a support system — tickets and responses, field mapping, re-sync
4	Audiences — who sees what
8	Best-practice checklist for good training data
10, 11	Every training endpoint, with curl / JavaScript / Python recipes

```
# smallest possible API example (secret key of the company)
curl -X POST https://port.smartpowerbilling.com/api/train/text \
  -H "Authorization: Bearer sk_live_xxx" -H "Content-Type: application/json" \
  -d '{"name":"Opening hours","content":"We are open Monday to Friday, 8:00-17:00.","audience":"public"}'
```

5 Sharing the Odoo add-on

A company that uses Odoo 17 gets the assistant inside Odoo by installing one add-on, **Brainbox AI** (`brainbox_ai`, version 17.0.2.0.0). You send them a zip; they install it and paste in a secret key. They do not copy any chat code: the chat window **is** the web SDK, bundled inside the add-on.

What to send

- `docs/brainbox_ai-odoo17-v2.0.0.zip` — the add-on folder (21 files, no caches). Or give them read access to a git repository containing only the `brainbox_ai` folder.
- A **secret key** for their tenant (section 2) — the add-on does not work with a publishable key for logged-in users, because only a secret key lets Odoo say who each user is and what role they have.
- The backend URL `https://port.smartpowerbilling.com` and their tenant id.

What the company does

- 1 Unzip into an addons folder.** The result must be `<addons path>/brainbox_ai/__manifest__.py`, in a folder listed in `addons_path` of their `odoo.conf` (Odoo.sh: commit the folder to their repository).
- 2 Restart Odoo.**
- 3 Apps → Update Apps List** (developer mode), search **Brainbox AI**, click **Activate**.
- 4 Settings → Brainbox AI:** fill in Backend URL, Tenant ID, Secret API key, click **Test connection**, choose a display mode, save.

The settings, briefly

Setting	What it is
Backend URL	<code>https://port.smartpowerbilling.com</code> (without <code>/api</code>).
Tenant ID	Their tenant id. Optional with a secret key (the key already says which tenant).
Secret API key (sk_...)	Kept on the Odoo server, never sent to browsers, never shown again. Empty = keep the stored key; "Remove it on save" deletes it.
SDK script URL	Optional: load the SDK from a URL (e.g. <code>https://port.smartpowerbilling.com/sdk/brainbox-web-sdk.js</code>) instead of the bundled copy.
Test connection	Button: the Odoo server calls <code>/api/health</code> with the stored key; it reports success, or a failure if the server cannot be reached or the key is rejected.
Display mode, Position	Floating launcher (default), docked sidebar, systray icon only, or disabled; bottom-right / bottom-left.
Allowed groups	Who gets the assistant. Empty = all internal users.
Role mapping, role groups	Which role (internal, customer, vendor, public...) each kind of user gets; decides which knowledge they may read.
Colors, names, logo, welcome message, quick actions	Appearance of the chat.

Every setting in detail, display modes, embedding the chat in a form and the role precedence: Odoo guide, section 7 "Installation and configuration" and section 5 "Who sees what". Onboarding checklist for a company: Odoo guide, section 6.

Updating the chat (SDK) inside the add-on

The add-on ships a copy of `sdk-web/brainbox-web-sdk.js` in `static/lib/brainbox-web-sdk/`. When the web SDK changes, copy it in, upgrade the module, and send a new zip:

```
# in odoo\custom_addons\brainbox_ai (PowerShell)
.\tools\sync_sdk.ps1 -SdkPath C:\Users\USER\Documents\PERSONAL\ai\sdk-web
# -Version 2 (download from npm via jsDelivr) only works once spres-web is published (section 7)
```

A company can also skip updates by setting **SDK script URL** to your server's copy, which every deploy keeps current. Bump the version in `__manifest__.py` and the zip name when you change the add-on, so companies can tell versions apart.

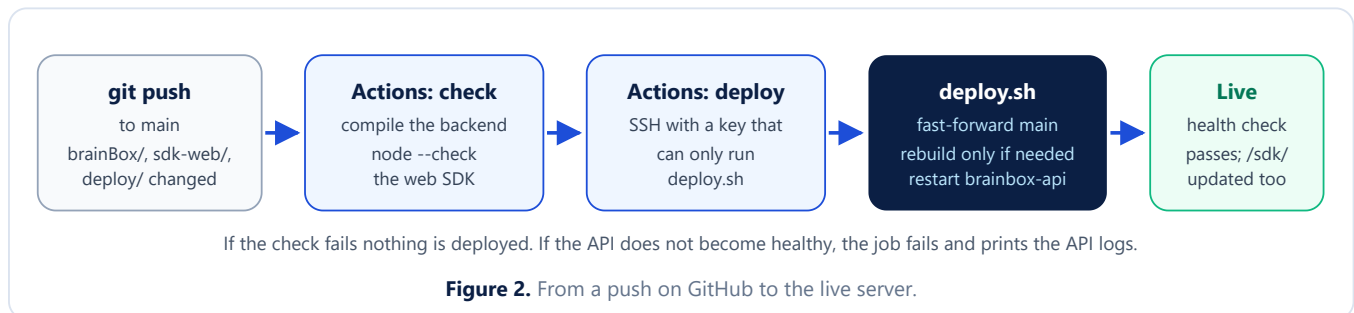
```
# rebuild the zip, without Python caches (PowerShell or cmd, in odoo\custom_addons)
tar -a -cf brainbox_ai-odoo17-v2.0.1.zip --exclude=__pycache__ --exclude=*.pyc brainbox_ai
```

The local Odoo test instance

```
cd C:\Users\USER\Documents\PERSONAL\odoo
# run Odoo: http://localhost:8069, database test_odoo_db (the only one with brainbox_ai installed)
.venv\Scripts\python.exe odoo-bin -c odoo.conf
# upgrade the add-on after changing it (add --http-port 8075 if 8069 is busy)
.venv\Scripts\python.exe odoo-bin -c odoo.conf -d test_odoo_db -u brainbox_ai --stop-after-init
```

6 Deployments and backups

You never copy files to the server by hand. You push to the `main` branch on GitHub; GitHub checks the code and tells the server to update itself. The whole thing takes about a minute.



- **Watch it:** GitHub → **Actions** tab → "Deploy brainBox". A red cross means look at the log of the failing step.
- **Re-run without a code change:** Actions → Deploy brainBox → **Run workflow**.
- **Postgres, Redis and Ollama keep running** during a deploy; only the API restarts (a few seconds of downtime). The API image is rebuilt only when `requirements.txt` or the `Dockerfile` changed.
- **Setup:** GitHub secrets `DEPLOY_HOST`, `DEPLOY_USER`, `DEPLOY_KNOWN_HOSTS`, `DEPLOY_SSH_KEY` and the forced-command line in the server's `authorized_keys` — see `deploy/README.md`.

Manual deploy

```
# on the server
bash /var/www/Spres-Ai-Agent/deploy/deploy.sh
FORCE=1 bash /var/www/Spres-Ai-Agent/deploy/deploy.sh
FORCE_BUILD=1 bash /var/www/Spres-Ai-Agent/deploy/deploy.sh

# same as GitHub does
# restart even if already up to date
# also rebuild the API image
```

`deploy.sh` only fast-forwards: if someone edited files directly on the server and committed there, it stops with an error instead of throwing their work away. Fix it by hand (`git status`, `git log`) before deploying again.

Database changes ("migrations")

There is **no separate migrate command**. Every time the API starts it runs `init_db`, which creates missing tables and adds missing columns (`CREATE ... IF NOT EXISTS`, `ADD COLUMN IF NOT EXISTS`). It is safe to run any number of times and never deletes data. A deploy therefore upgrades the database automatically. Take a backup before a big release anyway.

Backups

Only the Postgres database matters: it holds every company's knowledge, chats, keys and staff. Redis is a cache and the AI model can be downloaded again. Also keep a copy of `brainBox/.env` in your password manager — without `JWT_SECRET_KEY` everyone is logged out, without the rest you start over.

```
# on the server - one compressed dump file
mkdir -p /root/backups
docker exec brainbox-postgres pg_dump -U spres -Fc spres_ai > /root/backups/brainbox-$(date +%F).dump

# copy it off the server (from your laptop)
scp root@165.227.77.33:/root/backups/brainbox-2026-10-09.dump .

# optional: nightly at 03:00, keep 14 days (crontab -e)
0 3 * * * docker exec brainbox-postgres pg_dump -U spres -Fc spres_ai > /root/backups/brainbox-$(date +%F).dump && find /root/backups -name 'brainbox-*.dump' -mtime +14 -delete
```

Restore

Restoring **replaces** the whole database with the backup. Stop the API first so nothing writes in the meantime.

```
docker stop brainbox-api
docker exec brainbox-postgres dropdb -U spres spres_ai
docker exec brainbox-postgres createdb -U spres spres_ai
docker exec -i brainbox-postgres pg_restore -U spres -d spres_ai --no-owner < /root/backups/brainbox-2026-10-09.dump
docker start brainbox-api
curl -s http://127.0.0.1:8000/api/health
```

Test a restore on a spare server once, so you know the backups are good before you need them. After restoring an older backup the API adds any newer columns itself on start.

7 SDK releases

There are two ways an SDK reaches people. The **web SDK file** is served by your own server, so every deploy updates it. The **packages** (npm, PyPI) are what other developers install, and you publish those by hand when you decide a version is ready.

Folder	Package	Status (9 Oct 2026)	Notes
sdk-web	server copy <code>/sdk/brainbox-web-sdk.js</code>	LIVE	nginx serves it straight from the repo: every deploy publishes it. This is the URL to give websites today.
sdk-web	npm <code>spres-web</code> 2.0.0	PREPARED, NOT PUBLISHED	Never published: <code>npm view spres-web</code> → 404. The jsDelivr URLs in its README and the Odoo <code>sync_sdk.ps1 -Version</code> option need it.
sdk-react	npm <code>spres-react</code> 1.1.0	PREPARED, NOT PUBLISHED	npm has 1.0.4 (chat components only). 1.1.0 adds <code>TrainingPanel</code> , <code>StaffDashboard</code> and <code>BrainboxStaffClient</code> .
sdk-node	npm <code>spres-ai</code> 1.0.0	PUBLISHED	Server-side client (secret key).
sdk-python	PyPI <code>spres-ai</code> 1.0.0	PUBLISHED	Server-side client (secret key).
sdk-react-native	<code>brainbox-react-native-sdk</code> 1.0.0	OUTDATED	Its streaming chat calls <code>/api/chat/stream</code> , which does not exist on the server. Do not give it to anyone until it is updated to the current API (the React SDK is the reference).

Publishing an npm package

- 1 Bump the version** in `package.json`. npm refuses to publish a version number twice, ever (even after unpublishing).
`npm version patch` (1.1.0 → 1.1.1), **minor** for new features, **major** for breaking changes.
- 2 Build and check.** `npm run build` (sdk-react: TypeScript → `dist/`; sdk-web: minified file in `dist/`, run automatically by `prepublishOnly`).
- 3 Log in once** with the npm account that owns the packages: `npm login` (or an automation token in `~/.npmrc` — never commit that file).
- 4 Publish** and commit the version bump.

```
cd ai/sdk-react
npm install
npm run build
# list exactly which files will be uploaded: no .env, no keys
npm pack --dry-run
npm publish --access public

cd ../sdk-web
npm publish --access public # prepublishOnly runs: node --check + build
```

```
# Python (sdk-python): bump version in pyproject.toml, then
pip install build twine
python -m build
twine upload dist/*          # uses a PyPI API token; see PUBLISHING_GUIDE.md
```

Step-by-step account and token setup for npm and PyPI: [ai/PUBLISHING_GUIDE.md](#). After publishing `spres-web`, websites can also use <https://cdn.jsdelivr.net/npm/spres-web@2>, and the Odoo add-on's `sync_sdk.ps1 -Version 2` starts working.

8 Moving to a new server

Moving means: set up a fresh server with one script, copy the database across, point the domain at the new machine, and tell GitHub where to deploy. Plan about an hour, most of it waiting for downloads. Users notice only a short pause while DNS switches.

THE CURRENT SERVER IS SHARED

165.227.77.33 also runs unrelated services. On it, only ever touch the `brainbox-*` containers, the brainBox nginx site and `/var/www/Spres-Ai-Agent`.

Step by step

- 1 Create the new server.** Ubuntu 22.04 or 24.04, at least 4 vCPU and 8 GB RAM for `qwen2.5:1.5b` on CPU (more cores = faster answers; a GPU server for answers under 5 s), 40 GB+ disk. Log in with an SSH key.
- 2 Run the bootstrap script.** It installs Docker, clones the repo, writes a new `.env` with fresh secrets, starts the containers, downloads the model and waits for health.

```
curl -fsSL https://raw.githubusercontent.com/resbrotherx/Spres-Ai-Agent/main/deploy/bootstrap-server.sh -o bootstrap-server.sh
sudo bash bootstrap-server.sh https://github.com/resbrotherx/Spres-Ai-Agent.git
port.smartpowerbilling.com
```
- 3 Carry over the secrets that must not change.** Copy `JWT_SECRET_KEY` (keeps staff logged in), `BRAINBOX_ADMIN_TOKEN`, `DASHBOARD_URL`, `SMTP_*` and any tuning variables from the old `.env` into the new one. Keep the **new** `POSTGRES_PASSWORD` (the new database was created with it). Then `docker restart brainbox-api`.
API keys and staff passwords live in the database, so they move with the backup — companies change nothing.
- 4 Back up the old database and restore it on the new server** (section 6). For a clean cut-over, take the final backup right before the DNS switch.

```
# old server
docker exec brainbox-postgres pg_dump -U spres -Fc spres_ai > /root/backups/move.dump
# copy across (from the old server, or via your laptop)
scp /root/backups/move.dump root@NEW_IP:/root/move.dump
# new server: the restore commands from section 6 (stop api, dropdb, createdb, pg_restore, start api)
```
- 5 Test before switching.** On the new server `curl -s http://127.0.0.1:8000/api/health` and `curl -s -H "X-API-Key: sk_live_xxx" http://127.0.0.1:8000/api/health` with a real key (shows its tenant), then ask one question.
- 6 nginx and HTTPS.** Create the nginx site printed by the script (proxy to 127.0.0.1:8000, `location /sdk/`, `proxy_read_timeout 300s`), plus `location /dashboard/` if you host the dashboard (section 3).
- 7 Switch DNS.** Point the A record of `port.smartpowerbilling.com` to the new IP (lower the TTL to 300 a day before). Once it resolves to the new server, run `certbot --nginx -d port.smartpowerbilling.com`.

8 Update GitHub. Secrets `DEPLOY_HOST` (new IP) and `DEPLOY_KNOWN_HOSTS` (`ssh-keyscan -t ed25519 NEW_IP`); add the forced-command deploy key line to the new server's `~/.ssh/authorized_keys` (`deploy/README.md`). Then Actions → Run workflow to prove it works.

9 Firewall, backups, clean-up. Section 9 checklist, the nightly backup cron (section 6). Keep the old server stopped (not deleted) for a week, then remove its brainbox containers and volumes.

The AI model is not in the backup: the bootstrap script downloads the one named in `OLLAMA_MODEL` (by hand: `docker exec brainbox-ollama ollama pull <model>`). Changing model = edit `OLLAMA_MODEL`, pull it, restart the API.

Performance settings (answer speed)

Answers are written by the AI model running on the server's CPU, so speed depends mostly on the number of CPU cores, the model size and how much text goes into each prompt.

WHAT TO EXPECT TODAY

Production runs `qwen2.5:1.5b` on the shared 4-core CPU with the values in the "Production" column below. Measured: **about 9–20 s per answer**, depending on server load and answer length. Tuning cannot make a CPU much faster than that. **For answers under 5 s**, use a hosted OpenAI-compatible API (`LLM_PROVIDER=openai`, e.g. Groq `llama-3.1-8b-instant`: about 1–3 s, paid per use, questions leave your server) or move Ollama to a GPU server.

These variables in `brainBox/.env` exist in the code as of this writing (restart the API after changing them):

Variable	Production	Effect
<code>OLLAMA_MODEL</code>	<code>qwen2.5:1.5b</code>	The model. Smaller = faster, larger = better answers (<code>llama3.2:3b</code> is roughly twice as slow on the same CPU). Must be pulled into the ollama container. Same value in <code>.env.example</code> ; code fallback if unset: <code>llama2</code> .
<code>LLM_PROVIDER</code>	<code>ollama</code> (default)	<code>openai</code> = use any OpenAI-compatible API instead of the local model (much faster, costs money per question, questions leave your server). The other provider is tried as a fallback when configured.
<code>OPENAI_API_KEY</code> , <code>OPENAI_BASE_URL</code> , <code>OPENAI_MODEL</code>	not set (model default <code>gpt-4o-mini</code>)	For <code>LLM_PROVIDER=openai</code> (OpenAI, Groq, Together...; about 1–3 s per answer). Groq example: <code>OPENAI_BASE_URL=https://api.groq.com/openai/v1</code> , <code>OPENAI_MODEL=llama-3.1-8b-instant</code> . <code>USE_OPENAI=true</code> + a key enables it only as fallback.
<code>OLLAMA_NUM_CTX</code>	2048	Context window in tokens. Bigger = slower; keep it constant (changing it reloads the model).
<code>OLLAMA_NUM_PREDICT</code>	220	Maximum answer length in tokens. Lower = faster, shorter answers.
<code>OLLAMA_NUM_THREAD</code>	not set (0 = auto)	CPU threads for the model. Usually leave on auto; set to the number of physical cores if answers are slow.
<code>OLLAMA_KEEP_ALIVE</code>	-1 (forever)	How long the model stays in memory after a question. <code>-1</code> avoids the slow first question after a quiet period.
<code>OLLAMA_WARMUP</code>	true	Load the model when the API starts, so the first question after a deploy is not slow.
<code>LLM_TEMPERATURE</code>	not set (0.2)	Creativity. Low = factual and consistent.

Variable	Production	Effect
<code>LLM_TIMEOUT</code>	not set (120 s)	How long the API waits for the model before giving the "model unavailable" answer. Keep it below nginx's <code>proxy_read_timeout</code> (300 s) and the Odoo proxy's 200 s.
<code>RAG_MAX_CHUNKS</code> , <code>RAG_MAX_CHUNK_CHARS</code>	3, 450	How much knowledge text goes into each prompt. Prompt size dominates CPU time: fewer/shorter = faster, but less context.

The API logs one line per answer: `Ollama answered in 9.3s (load 0.0s, prompt 812 tok, answer 140 tok)`. A large *load* means the model was reloaded; a large *prompt* means lower the RAG settings. `docker stats` shows whether the CPU is saturated.

9 Security checklist

Go through this list after every server change, and once a quarter anyway.

Network

- ☐ **Only ports 22, 80 and 443 are open to the internet.** `ufw allow 22/tcp, 80/tcp, 443/tcp`, default deny incoming — and/or the same rules in the cloud provider's firewall (DigitalOcean → Networking → Firewalls).
- ☐ **Port 8000 is not reachable from outside.** Docker publishes container ports past `ufw`, and `docker-compose.yml` maps the API as `8000:8000` on all interfaces. Block 8000 in the cloud firewall, or change the mapping to `"127.0.0.1:8000:8000"` (nginx reaches it locally either way). Test from your laptop: `curl -m 5 http://165.227.77.33:8000/api/health` must time out.
- ☐ **Postgres, Redis and Ollama are never published.** Postgres is bound to `127.0.0.1:5432` (use an SSH tunnel to inspect it); Redis and Ollama have no `ports:` at all. Keep it that way: Redis and Ollama have no password.
- ☐ **HTTPS only.** certbot installed with automatic renewal (`certbot renew --dry-run` succeeds).

Access

- ☐ **SSH keys, not passwords.** In `/etc/ssh/sshd_config`: `PasswordAuthentication no` (and ideally `PermitRootLogin prohibit-password`), then `systemctl reload ssh` — only after confirming your key login works in a second terminal.
- ☐ **The GitHub deploy key is restricted** to `command="bash /var/www/Spres-Ai-Agent/deploy/deploy.sh"` with `restrict`, and is used for nothing else.
- ☐ **No server password is written down** in the repo, in documents or in chats. Secrets live only in the server's `.env` and your password manager.
- ☐ **The repository is public:** never commit `.env`, keys, dumps or `.npmrc` tokens. Before every commit, `git status` and look.

Secrets and keys

- ☐ **Secret keys (sk_) only on servers** (Odo, back-ends, scripts). Websites and mobile apps get publishable keys (pk_) or go through a server proxy.
- ☐ **BRAINBOX_ADMIN_TOKEN and JWT_SECRET_KEY are long random values** (the bootstrap script generates 64 hex characters) and the startup log shows no placeholder warning.
- ☐ **Rotate** the admin token if it may have been seen (change `.env`, restart the API). Changing `JWT_SECRET_KEY` logs out every staff member — do it if a staff session may have been stolen.
- ☐ **REQUIRE_API_KEY=true** (the default). `false` is a migration-only kill switch that turns off key checks for everyone.
- ☐ **DEBUG=false** in production.
- ☐ **Unused keys revoked:** `keys list` shows *last used*; revoke keys of companies that left and keys that have not been used for months. Deactivate staff who left.
- ☐ **Backups exist off the server**, and a restore has been tested (section 6). Backups contain every company's data: store them encrypted.

10 Troubleshooting

Start with the three health checks, then the logs. Almost every problem shows up in `docker logs brainbox-api`.

First look

```
# from anywhere: through nginx + HTTPS
curl -s https://port.smartpowerbilling.com/api/health
# on the server: straight to the API (skips nginx)
curl -s http://127.0.0.1:8000/api/health
# is a key valid, and for which tenant?
curl -s -H "X-API-Key: sk_live_xxx" https://port.smartpowerbilling.com/api/health
# database, cache and vector checks (secret key)
curl -s -H "X-API-Key: sk_live_xxx" https://port.smartpowerbilling.com/api/health/db

# on the server
docker ps --filter name=brainbox          # all four Up? postgres/redis "(healthy)"?
docker logs --tail 100 brainbox-api       # add -f to follow live
docker logs --tail 50 brainbox-ollama
docker stats --no-stream                  # CPU / memory per container
df -h /                                   # disk full breaks everything
```

If `127.0.0.1:8000` works but the public URL does not, the problem is nginx or DNS (`nginx -t`, `systemctl status nginx`, `/var/log/nginx/error.log`). A **502** from nginx means the API container is down; a **504** means it answered too slowly.

Symptoms

Symptom	Likely cause	Fix
401 "Invalid or missing API key"	No key sent, typo, revoked or expired key.	<code>keys list --tenant X</code> : is it active? Send it as <code>X-API-Key</code> or <code>Authorization: Bearer</code> . Create a new one if lost (raw keys cannot be shown again).
403 "This endpoint requires a secret API key"	A publishable key used for training, ingest or detailed health.	Use the company's secret key on a server, or a staff login.
403 "tenant_id does not match API key"	The request names another tenant than the key's.	Remove <code>tenant_id</code> (the key decides) or fix the typo, e.g. in Odoo's Tenant ID.
403 "Staff login required"	An API key used on a dashboard endpoint (<code>/api/staff</code> , <code>/api/keys</code> , <code>/api/reports</code> ...).	These need a staff login (email + password).
403 "Your role (viewer) can't do this"	Staff role too low.	An admin/owner raises the role in Staff.
429 on staff login	Too many failed logins (10 per 15 min per email + IP).	Wait 15 minutes, or reset with <code>staff set-password</code> .
401 / 503 on <code>/api/admin/*</code>	Wrong <code>X-Admin-Token</code> (401) or <code>BRAINBOX_ADMIN_TOKEN</code> not set (503).	Check the token in <code>.env</code> ; or use the CLI, which needs no token.
Answer: "the AI model service is currently unavailable"	Ollama down, model not pulled, or the model took longer than <code>LLM_TIMEOUT</code> .	<code>docker exec brainbox-ollama ollama list</code> ; if the model named in <code>OLLAMA_MODEL</code> is missing: <code>docker exec brainbox-ollama ollama pull <model></code> . Check <code>docker logs brainbox-api grep -i ollama</code> .

Symptom	Likely cause	Fix
Slow answers	Normal on CPU: 9–20 s with <code>qwen2.5:1.5b</code> on the shared 4-core server. Worse when the model reloads, prompts are large or other services load the CPU.	Check the "Ollama answered in ..." log line and <code>docker stats</code> ; keep the section 8 values. For under 5 s: a hosted LLM (<code>LLM_PROVIDER=openai</code> , e.g. Groq <code>llama-3.1-8b-instant</code> , 1–3 s) or a GPU server.
First answer after a deploy is slow	The model is loaded into memory on first use.	Keep <code>OLLAMA_WARMUP=true</code> and <code>OLLAMA_KEEP_ALIVE=-1</code> .
"I don't have that information yet"	No matching knowledge for that user's audience.	Normal: it appears under Knowledge gaps. Train it, or check the source's audience (website = public).
Invite email never arrives	SMTP not configured or rejected.	Copy the invite link from the dashboard; check <code>SMTP_*</code> and <code>docker logs brainbox-api grep -i smtp</code> .
Invite link opens the wrong page	<code>DASHBOARD_URL</code> unset or wrong.	Set it to the hosted dashboard (section 3), restart the API, resend the invite.
GitHub deploy failed	Check job: syntax error. Deploy job: SSH (host key / key) or health check.	Read the failing step's log; on the server run <code>bash /var/www/Spres-Ai-Agent/deploy/deploy.sh</code> by hand to see the same output.
API keeps restarting	Database password mismatch, syntax error, missing package.	<code>docker logs brainbox-api</code> ; after a <code>requirements.txt</code> change: <code>FORCE_BUILD=1 bash deploy/deploy.sh</code> .
Odoo: "Cannot reach the Brainbox AI server"	Wrong Backend URL, server down, or firewall between Odoo and the server.	Settings → Brainbox AI → Test connection; <code>curl</code> the health URL from the Odoo machine.

Restarting things

```
# safe at any time (a few seconds of downtime)
docker restart brainbox-api
# start whatever is missing
cd /var/www/Spres-Ai-Agent/brainBox && docker compose up -d postgres redis ollama api
# the model reloads on the next question
docker restart brainbox-ollama
```

11 Quick reference card

Connect

```
ssh root@165.227.77.33
cd /var/www/Spres-Ai-Agent/brainBox
# CLI prefix used below:
CLI="docker exec brainbox-api python -m app.cli"
```

Health

```
curl -s https://port.smartpowerbilling.com/api/health
curl -s http://127.0.0.1:8000/api/health
curl -s -H "X-API-Key: sk_live_xxx" \
https://port.smartpowerbilling.com/api/health
```

Staff

```
$CLI staff create --tenant acme \
--email owner@acme.com --role owner # invite link
$CLI staff list --tenant acme
docker exec -it brainbox-api python -m app.cli \
staff set-password owner@acme.com
$CLI staff deactivate someone@acme.com
```

Keys (raw key shown once)

```
$CLI keys create --tenant acme \
--type publishable --name Website
$CLI keys create --tenant acme \
--type secret --name Odoo
$CLI keys list --tenant acme
$CLI keys revoke 42
```

Admin API (X-Admin-Token)

```
POST /api/admin/staff {tenant_id,email,role}
POST /api/admin/keys {tenant_id,key_type,name}
GET /api/admin/keys?tenant_id=acme
DELETE /api/admin/keys/{id}
```

Deploy

```
git push origin main # automatic
bash /var/www/Spres-Ai-Agent/deploy/deploy.sh
FORCE=1 bash ... /deploy/deploy.sh # restart
FORCE_BUILD=1 bash ... /deploy/deploy.sh # rebuild
```

Logs and containers

```
docker ps --filter name=brainbox
docker logs --tail 100 -f brainbox-api
docker stats --no-stream
docker restart brainbox-api
```

AI model

```
docker exec brainbox-ollama ollama list
grep OLLAMA_MODEL .env # the configured model
docker exec brainbox-ollama ollama pull qwen2.5:1.5b
docker restart brainbox-api # after changing it
```

Backup and restore

```
# backup
docker exec brainbox-postgres pg_dump -U spres -Fc spres_ai > /root/backups/brainbox-$(date +%F).dump
# restore (replaces everything)
docker stop brainbox-api
docker exec brainbox-postgres dropdb -U spres spres_ai && docker exec brainbox-postgres createdb -U spres spres_ai
docker exec -i brainbox-postgres pg_restore -U spres -d spres_ai --no-owner < FILE.dump && docker start brainbox-api
```

Settings file

```
nano /var/www/Spres-Ai-Agent/brainBox/.env
docker restart brainbox-api # apply changes
# never paste its contents anywhere
```

Odoo (local test)

```
cd C:\Users\USER\Documents\PERSONAL\odoo
.venv\Scripts\python.exe odoo-bin -c odoo.conf
# http://localhost:8069 · DB test_odoo_db
```

Dashboard (local)

```
cd ai\sdk-react\preview && npm run dev
# http://localhost:4173/#/staff
# mock data: ?mock=1#/staff/overview
```

Publish SDKs

```
npm version minor && npm run build
npm pack --dry-run && npm publish --access public
# PyPI: python -m build && twine upload dist/*
```

New server

```
sudo bash deploy/bootstrap-server.sh \
https://github.com/resbrotherx/Spres-Ai-Agent.git \
port.smartpowerbilling.com
```

Which credential where

pk_ websites, apps, sdk-web, ChatWidget · **sk_** Odoo, servers, sdk-node / sdk-python, training · staff login → dashboard · admin token → you only.

Never send: admin token, JWT secret, **.env**, DB password, SSH credentials.